



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Laurea Magistrale: Informatica

Report : Penetration Testing and Ethical Hacking

S.I.D.E. (SCADA Intrusion Detection Environment)

Studente: Umberto Della Monica

Matricola: 0522501617

Professore : Arcangelo Castiglione

Data: 9 novembre 2025

Sommario

Il progetto S.I.D.E. (SCADA/ICS Industrial Defense Environment) propone un ambiente di simulazione completo per testare e valutare la sicurezza dei sistemi industriali. Integrando simulatori per protocolli Modbus TCP, Siemens S7 e OPC-UA. S.I.D.E. permette di generare traffico realistico tra server e client, includendo comportamenti normali e anomalie controllate. Il sistema supporta inoltre l'analisi delle comunicazioni e l'integrazione con tecniche di rilevazione delle intrusioni, fornendo un framework flessibile per la sperimentazione di metodologie di difesa attiva, monitoraggio semantico e validazione di scenari di attacco realistici. Grazie alla combinazione di simulazione di dispositivi, raccolta dati e logging avanzato, S.I.D.E. rappresenta uno strumento utile per la ricerca, la formazione e la valutazione della resilienza dei sistemi ICS/SCADA.

Indice

1	Introduzione	3
1.1	Struttura del documento	3
2	Related Work	5
3	S.I.D.E. - Architecture	7
3.1	Architettura Generale	7
3.2	side-sniff	7
3.2.1	Obiettivi funzionali	8
3.2.2	Panoramica dell' IDS	8
3.2.3	Procedura operativa (step-by-step)	9
3.2.4	Decodifica dei Payload Applicativi	13
3.2.5	Modulo di Anomaly Detection: Z-Score Service	20
3.2.6	Modulo di Anomaly Detection: ML - Isolation Forest	22
3.2.7	Flusso di Comunicazione: side-sniff e side-device-discovery	24
3.3	side-device-discovery	25
3.3.1	Funzionamento generale	25
3.3.2	Struttura del codice	26
3.3.3	Flusso operativo	27
3.3.4	Proprietà principali	27
3.4	side-api-backend	27
3.4.1	Struttura generale	28
3.4.2	Punto di avvio: <code>run.py</code>	28
3.4.3	Gestione del database Neo4j	28
3.4.4	Gestione degli eventi asincroni: <code>service_discovery_consumer.py</code>	29
3.4.5	Gestione connessioni WebSocket: <code>websocket_manager.py</code>	29
3.4.6	Controller REST e WebSocket	30
3.4.7	Integrazione dei servizi e funzionamento complessivo	30
3.4.8	Sintesi del flusso operativo	30
3.4.9	Ruolo architetturale	31
3.4.10	Flusso di Interazione con il Backend	31

3.5	side-frontend	31
3.5.1	Dashboard interattiva	32
3.5.2	Sezione /devices	32
3.5.3	Sezione /events	33
3.5.4	Avvio del modulo	34
3.6	side-ml-models	34
3.6.1	Estrazione dei dati dai file PCAP	35
3.6.2	Addestramento del modello Isolation Forest	35
3.6.3	Osservazioni	36
3.7	Device Simulators	36
3.7.1	Modbus TCP	36
3.7.2	Siemens S7 (Snap7)	37
3.7.3	OPC-UA	38
3.7.4	Conclusioni sui simulatori	39
4	SCADA – Cyber Attacks	40
4.1	Introduzione	40
4.2	Metodo di riproduzione del traffico	40
4.3	Obiettivi sperimentali	40
4.4	Tipologie d’attacco considerate	41
4.4.1	Man-In-The-Middle (MITM)	41
4.4.2	modbusQuery2Flooding	41
4.4.3	modbusQueryFlooding	41
4.4.4	pingFloodDDoS (ICMP)	41
4.4.5	tcpSYNFloodDDoS	41
4.5	Organizzazione dei test e note pratiche	42
5	Conclusioni	43

1. Introduzione

Negli ultimi anni, la crescente interconnessione dei sistemi industriali e dei Supervisory Control and Data Acquisition (SCADA) ha esposto le infrastrutture critiche a minacce informatiche sempre più sofisticate. La ricerca ha dimostrato l'efficacia di approcci multilivello e semantici per la protezione dei sistemi ICS, l'utilizzo di tecniche di Machine Learning per l'identificazione delle anomalie e la simulazione di scenari realistici per la validazione delle difese esistenti [1, 11, 3, 6].

Il progetto S.I.D.E. nasce dall'esigenza di disporre di un ambiente controllato che permetta di testare strumenti di difesa, metodologie di rilevazione delle intrusioni e strategie di monitoraggio avanzate. A tal fine, S.I.D.E. integra simulatori di protocolli industriali (Modbus TCP, Siemens S7 e OPC-UA), consentendo la creazione di traffico client-server realistico, l'inserimento di anomalie programmate e la raccolta di dati dettagliati per analisi successive.

L'obiettivo principale è fornire un framework modulare e scalabile, utile per l'ambito industriale, che permetta di:

- simulare scenari ICS/SCADA realistici e variabili;
- valutare l'efficacia di sistemi di rilevazione delle intrusioni e meccanismi di difesa attiva;
- analizzare il comportamento dei protocolli industriali e delle architetture SCADA in condizioni normali e anomale;

1.1 Struttura del documento

Il presente documento è articolato in diversi capitoli, ognuno dei quali approfondisce aspetti specifici del lavoro svolto.

- **Capitolo 2** introduce e analizza i principali lavori correlati riguardanti la sicurezza dei sistemi *ICS/SCADA*, con particolare attenzione ai modelli di difesa basati su *Intrusion Detection Systems (IDS)*, approcci di *machine learning*, analisi semantica e metodologie di difesa attiva che rappresentano l'evoluzione delle strategie di protezione delle infrastrutture critiche.

- **Capitolo 3** descrive in dettaglio l'organizzazione del progetto, la struttura architeturale, i vari *sub-moduli* sviluppati e le tecnologie utilizzate, con particolare attenzione ai protocolli SCADA oggetto di studio.
- **Capitolo 4** riassume le conclusioni emerse dal lavoro, proponendo possibili miglioramenti e sviluppi futuri del sistema.

2. Related Work

Negli ultimi anni la sicurezza dei sistemi Industrial Control Systems (ICS) e Supervisory Control and Data Acquisition (SCADA) è diventata una priorità cruciale, spingendo la ricerca verso lo sviluppo di strumenti e metodologie per la rilevazione e la mitigazione delle minacce informatiche. Diversi lavori hanno contribuito alla definizione di ambienti di simulazione e modelli di difesa integrati, con l'obiettivo di testare e migliorare la resilienza dei sistemi critici. Tra questi, alcuni studi iniziali hanno proposto l'implementazione e la simulazione di servizi di sicurezza in infrastrutture SCADA reali e virtualizzate, evidenziando l'importanza di valutare il comportamento dei sistemi sotto attacco e la risposta dei meccanismi di difesa in scenari realistici [1].

Ulteriori ricerche hanno introdotto modelli multilivello di protezione, come il sistema a triplo livello per la sicurezza SCADA in reti elettriche, basato sull'interazione coordinata tra livelli fisici, logici e applicativi [5]. Questo approccio, pensato per le infrastrutture critiche, ha aperto la strada a nuove architetture difensive che combinano monitoraggio continuo, gestione delle anomalie e risposta automatica agli incidenti. Parallelamente, altri studi si sono concentrati sull'architettura di sicurezza informatica per la smart grid, sottolineando la necessità di una progettazione modulare che consenta la protezione di ogni componente del sistema [4].

Un contributo significativo è rappresentato dal progetto PRECINCT, che ha proposto un ecosistema di cooperazione pubblico-privato per migliorare la resilienza delle infrastrutture critiche urbane attraverso la condivisione dei dati e la correlazione degli eventi di sicurezza [16]. Tale approccio è stato affiancato da sistemi specifici per la rilevazione delle intrusioni, come DIDEROT IDS, un framework per la difesa in ambienti industriali basato su tecniche di Machine Learning e su moduli di analisi del traffico [11].

L'analisi dei pacchetti e l'identificazione dei dispositivi industriali tramite il payload dei messaggi rappresentano un'altra direzione di ricerca rilevante, volta a migliorare la consapevolezza del sistema e la profilazione dei dispositivi connessi [19]. Parallelamente, l'esplorazione della diversità nelle architetture SCADA ha permesso di comprendere meglio le vulnerabilità e le possibili superfici d'attacco [18].

Sono stati inoltre sviluppati approcci basati su regole per il monitoraggio delle reti SCADA [2] e strumenti come Suricata, che, attraverso l'estensione delle proprie funzionalità, si sono dimostrati efficaci nel rafforzare la sicurezza informatica in contesti ICS

[8]. Altri lavori hanno applicato metodi di Machine Learning supervisionato per la rilevazione delle intrusioni, mostrando come la classificazione automatica degli eventi possa incrementare la capacità di identificare comportamenti anomali [12].

A questi studi si aggiungono ricerche che propongono un approccio fisico-basato alla rilevazione delle intrusioni, combinando metriche di rete e misure fisiche dei processi per migliorare l'accuratezza degli IDS e ridurre i falsi positivi nei sistemi SCADA [10].

In aggiunta, framework specifici hanno integrato la semantica dei processi industriali per migliorare la precisione nella rilevazione delle minacce e ridurre i falsi positivi [7, 3]. Tali approcci semantici hanno consentito di analizzare le relazioni tra dati di processo e attività di rete, fornendo una visione più completa della sicurezza operativa.

Recentemente, la ricerca si è spostata verso l'uso di tecniche di difesa attiva e modelli basati su grafi, capaci di rappresentare le relazioni tra componenti di rete e potenziali vettori d'attacco [6]. In particolare, l'applicazione di Graph Neural Network (GNN) ha mostrato risultati promettenti nella classificazione e nella previsione di eventi di sicurezza nei sistemi elettrici industriali [17].

Anche il tema della valutazione del rischio nei PLC (Programmable Logic Controller) è stato affrontato con approcci quantitativi e metodologie per l'identificazione delle vulnerabilità [15]. Infine, sono emersi filoni di ricerca che combinano l'intelligenza artificiale con la sicurezza informatica in domini specifici, come l'agricoltura intelligente [13] e la protezione dei sistemi SCADA mediante tecniche AI avanzate [20], confermando la tendenza verso l'adozione di strumenti intelligenti per la difesa delle infrastrutture critiche.

3. S.I.D.E. - Architecture

Il sistema S.I.D.E. (SCADA Intrusion Detection Environment) è strutturato in moduli indipendenti ma interconnessi, ciascuno responsabile di una parte specifica del flusso di raccolta, analisi e visualizzazione dei dati. L'obiettivo principale è la rilevazione di anomalie e attacchi in ambienti SCADA, attraverso un'architettura a microservizi basata su messaggistica, analisi di pacchetti e modellazione comportamentale.

3.1 Architettura Generale

L'architettura complessiva si articola nei seguenti componenti principali:

- **side-sniff**: modulo di sniffing e analisi del traffico SCADA;
- **side-device-discovery**: servizio di scoperta e registrazione dei dispositivi in Neo4j;
- **side-api-back-end**: API REST per l'interazione tra backend, database e frontend;
- **side-frontend**: interfaccia grafica per la visualizzazione e il monitoraggio realtime;
- **side-ml-models**: componente di machine learning integrato nello sniffer;
- **Device Simulators**: insieme di dispositivi SCADA simulati (Modbus, OPC-UA, Snap7).

3.2 side-sniff

Il modulo **side-sniff** rappresenta il core del sistema: un *IDS* modulare e cross-platform per la cattura, l'analisi e la classificazione del traffico SCADA/ICS. Il design è pensato per essere leggero in fase di cattura ma estendibile nelle componenti di analisi, con particolare attenzione a prestazioni, robustezza e facilità di estensione per nuovi protocolli.

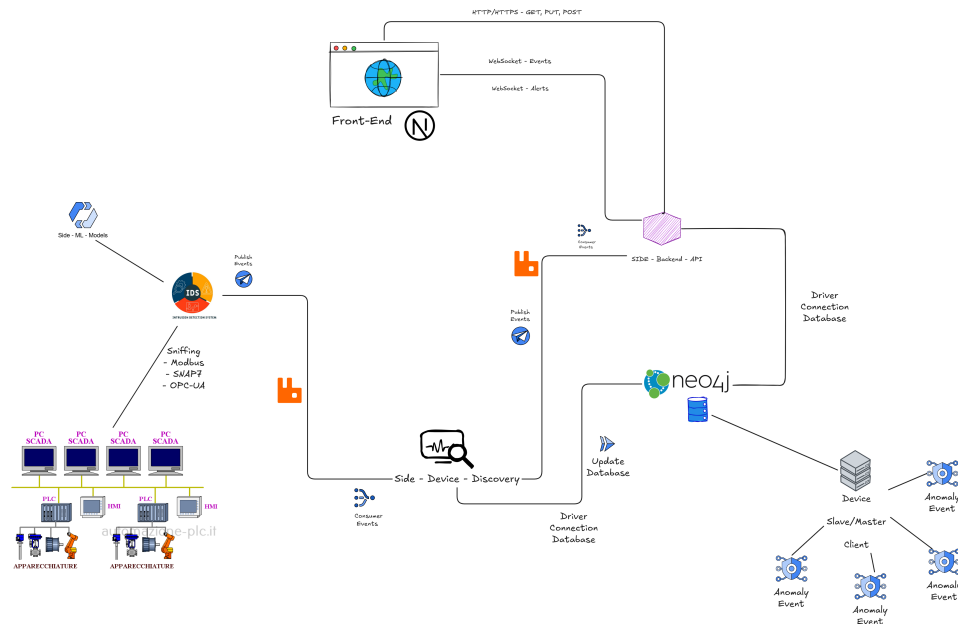


Figura 3.1: Struttura concettuale dell'Architettura SCADA

3.2.1 Obiettivi funzionali

- Cattura passiva del traffico su interfacce di rete (Linux, Windows, macOS).
- Parsificazione multilivello (IP, Trasporto) con dissector specifici per Modbus, OPC-UA, Snap7.
- Rilevamento multi-livello: firme (rule matching), rilevamento statistico (Z-scoring) e apprendimento automatico (classificazione/anomalia).
- Pubblicazione dei vari messaggi di alert mediante protocollo MQTT per tipologia di Protocollo gestito.

3.2.2 Panoramica dell' IDS

Il servizio opera come *daemon/service* e si compone di tre blocchi principali:

1. **Capture layer**: gestisce la cattura del traffico di rete attraverso librerie compatibili con `libpcap` / `Npcap`, utilizzate internamente da `Scapy`. È responsabile dell'acquisizione dei pacchetti e dell'applicazione di eventuali filtri BPF per limitare il traffico analizzato.
2. **Dispatcher**: riceve i pacchetti grezzi catturati, effettua il parsing dei layer di rete e trasporto (IP, TCP/UDP) e identifica il protocollo SCADA/ICS associato tramite la *protocol tree*.

3. **Handler:** contiene i parser specifici per i protocolli industriali supportati (Modbus, Snap7, OPC-UA) e i moduli di analisi e rilevamento (rule-based, statistico, o basato su machine learning).

3.2.3 Procedura operativa (step-by-step)

La seguente sequenza descrive il flusso operativo interno del modulo `side-sniff`, coerente con l'implementazione attuale del prototipo Python.

1. **Avvio e scelta dell'interfaccia:** all'avvio, il programma può essere eseguito in modalità *interactive* (CLI/TUI) oppure come *daemon*, con l'interfaccia già specificata nel file di configurazione. In modalità interattiva, l'utente seleziona l'interfaccia di rete su cui effettuare la cattura, tramite la funzione `select_interface()` che interroga le API di sistema fornite da `libpcap/Npcap` (via `Scapy`). La selezione può includere un filtro BPF opzionale, che verrà applicato durante la cattura per ridurre il traffico non rilevante.

```

(side-env) PS C:\Users\umber\Progetti\SIDE\side-sniff> python .\main.py
2025-11-04 18:52:01,487 - INFO - 17 regole caricate dal file JSON.
[*] Sistema operativo rilevato: Windows
[*] Interfacce disponibili:
0: \Device\NPF_{0028B026-CBA9-4DCD-B818-303AA50E0004} | IP: 0.0.0.0 | MAC: 00:00:00:00:00:00 | Status: N/A | MTU: N/A | Speed: N/A
1: \Device\NPF_{9985F7A1-CFFF-4CEA-A831-3A2E76B75174} | IP: 0.0.0.0 | MAC: 00:00:00:00:00:00 | Status: N/A | MTU: N/A | Speed: N/A
2: \Device\NPF_{A9EA4C56-6EA2-4990-96D0-488204D94906} | IP: 0.0.0.0 | MAC: 00:00:00:00:00:00 | Status: N/A | MTU: N/A | Speed: N/A
3: \Device\NPF_{924D2912-F40C-477F-A410-1EB1C13C8487} | IP: 192.168.144.1 | MAC: 00:15:5d:71:92:d9 | Status: N/A | MTU: N/A | Speed: N/A
4: \Device\NPF_{90750F70-7923-4F0F-9925-57E8C38CD7FC} | IP: 192.168.1.18 | MAC: 26:c7:fb:f9:39:36 | Status: N/A | MTU: N/A | Speed: N/A
5: \Device\NPF_{E93E58F1-E21F-49F9-AE8D-986A100A787B} | IP: 192.168.199.1 | MAC: 00:50:56:c0:00:08 | Status: N/A | MTU: N/A | Speed: N/A
6: \Device\NPF_{13CAF1A2-AAC8-47CC-98F9-E924BA3C50E4} | IP: 192.168.197.1 | MAC: 00:50:56:c0:00:01 | Status: N/A | MTU: N/A | Speed: N/A
7: \Device\NPF_{46F93B9C-524E-4729-B154-3E6380F8161A} | IP: 169.254.7.155 | MAC: d6:d2:52:82:c6:39 | Status: N/A | MTU: N/A | Speed: N/A
8: \Device\NPF_{B08CF4BA-4503-4F93-B555-5FD9F02200B7} | IP: 169.254.201.132 | MAC: d4:d2:52:82:c6:3a | Status: N/A | MTU: N/A | Speed: N/A
9: \Device\NPF_{Loopback} | IP: 127.0.0.1 | MAC: 00:00:00:00:00:00 | Status: N/A | MTU: N/A | Speed: N/A
10: \Device\NPF_{66E8C420-C65E-461F-B1C4-088D32B0658A} | IP: 169.254.184.145 | MAC: 04:d4:c4:79:e3:aa | Status: N/A | MTU: N/A | Speed: N/A
Seleziona il numero dell'interfaccia da usare (default 0):

```

Figura 3.2: Schermata di selezione dell'interfaccia in `side-sniff`

2. **Inizializzazione del Capture layer:** una volta scelta l'interfaccia, il modulo `core.side_sniffer_core` avvia un thread dedicato alla cattura tramite la funzione `run_sniffer_in_thread()`. Tale funzione utilizza `scapy.sniff()` per aprire la sessione di cattura, specificando:
 - l'interfaccia selezionata (`iface`);
 - la funzione di callback `dispatch_packet` per l'elaborazione di ogni pacchetto;
 - l'eventuale filtro BPF (al momento lasciato vuoto nel prototipo);
 - una funzione di terminazione che consente di arrestare lo sniffer in modo controllato tramite segnale `KeyboardInterrupt`.
3. **Acquisizione e dispatch dei pacchetti:** i pacchetti grezzi acquisiti vengono passati direttamente al `Dispatcher` attraverso la callback `dispatch_packet()`. Quest'ultimo effettua il parsing dei layer di trasporto (TCP/UDP), identifica la porta

sorgente o destinazione e consulta la struttura `protocol_tree` per associare il pacchetto a un protocollo noto. In caso di corrispondenza, il pacchetto viene inoltrato al rispettivo *handler* (es. `modbus_packet_handler()`, `s7_packet_handler()`, `opcua_packet_handler()`), che si occupa della successiva fase di analisi.

4. **Identificazione tramite Protocol Tree:** la componente `protocol_tree` costituisce la logica di identificazione dei protocolli SCADA/ICS all'interno dello sniffer. Tale struttura dati è implementata mediante una *Radix Tree* (o *Patricia Trie*), progettata per eseguire ricerche efficienti su chiavi testuali compatte come "TCP-502" o "UDP-4840", che rappresentano le coppie *layer-porta*.

SCADA Protocol Management

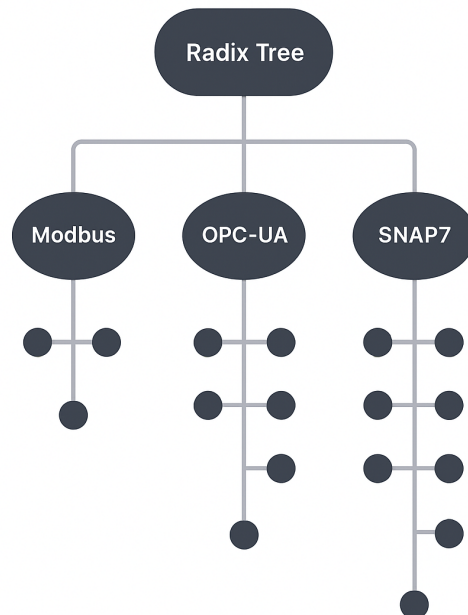


Figura 3.3: Struttura concettuale della *Protocol Radix Tree* di `side-sniff`

Durante l'inizializzazione, il modulo `core.side_sniffer_protocol_tree` importa la classe `RadixTreeProtocolFilter`, la quale:

- costruisce la Radix Tree popolandola con regole caricate da un file JSON esterno (`rules.json`);
- associa a ciascun nodo terminale un insieme di regole che definiscono un protocollo (es. Modbus, S7, OPC-UA);

- consente ricerche rapide basate su chiavi di layer e porta, restituendo le regole o i metadati associati al protocollo.

Il caricamento delle regole avviene tramite la funzione:

```
protocol_tree.load_from_json(f"{BASE_DIR_RULES}/{RULES_FILE}")
```

dove il file JSON descrive in forma compatta le associazioni tra layer, porta e protocollo:

Listing 3.1: Formulazione delle regole generali in JSON - SCADA

```
[
  { "layer": "TCP", "port": 502, "protocol": "Modbus" },
  { "layer": "TCP", "port": 102, "protocol": "S7" },
  { "layer": "TCP", "port": 4840, "protocol": "OPC-UA" }
]
```

In fase di analisi, la funzione `dispatch_packet()` costruisce una chiave "`<layer>-<porta>`" e invoca:

```
match = protocol_tree.search(key)
```

Se viene trovata una corrispondenza, il protocollo viene riconosciuto e il pacchetto è inoltrato all'*handler* specifico.

5. **Parsing e gestione dei protocolli SCADA:** una volta che il Dispatcher riceve i pacchetti raw dal livello di cattura, essi vengono indirizzati ai moduli *Handler* appropriati, ciascuno specializzato nell'analisi di un protocollo specifico (Modbus/TCP, S7, OPC-UA). Ogni handler esegue il parsing del payload, estrae campi significativi per il livello applicativo e applica controlli di validità e rilevamento anomalie basati su modelli statistici e di *machine learning*.

Handler Modbus/TCP Il modulo `modbus_packet_handler` è responsabile dell'elaborazione dei pacchetti TCP diretti alla porta 502. Dopo la verifica dei layer IP e TCP, l'handler estrae il payload grezzo (**Raw**) e lo passa alla funzione di decodifica `parse_modbus_payload()`, che interpreta la struttura MBAP (Transaction ID, Protocol ID, Length, Unit ID) e la Function Code. In caso di eccezione, viene generato un log diagnostico e registrato l'evento. Ogni pacchetto viene poi convertito in un vettore di *feature* (Function Code, lunghezza del campo dati, presenza di eccezioni) e analizzato dal servizio `ZScoreService`. Se viene rilevata un'anomalia statistica,

il controllo passa al modello ML `IsolationForestModbusService`, che conferma o smentisce la natura anomala del pacchetto. Gli eventi anomali vengono pubblicati nel sistema di monitoraggio con metadati completi (IP sorgente/destinazione, Function Code, nome funzione, dati in esadecimale).

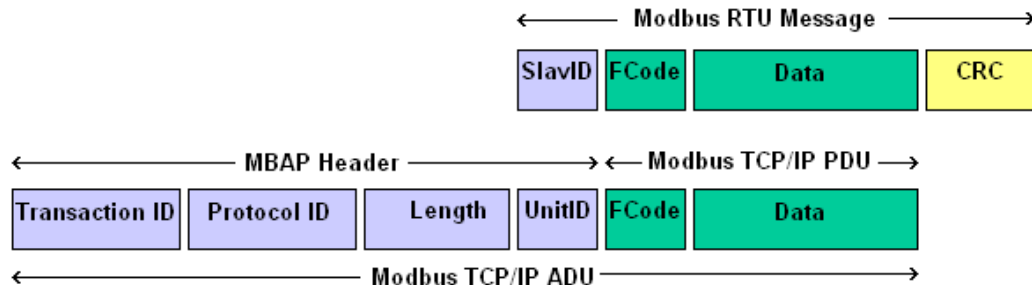


Figura 3.4: Struttura di un pacchetto Modbus/TCP con header MBAP e PDU applicativo.

Handler S7 (Siemens S7Comm) L'handler `s7_packet_handler` gestisce i pacchetti TCP destinati alla porta 102. Effettua un parsing multilivello (TPKT → COTP → S7Comm) decodificando i campi fondamentali: versione e lunghezza TPKT, tipo PDU COTP, codice ROSCTR, riferimento PDU e parametri applicativi. I campi `job_function`, `area_name`, `db_number`, `parameters` e `data` vengono interpretati e convertiti in formato leggibile (ASCII) per la successiva analisi. Anche in questo caso le *feature* (funzione, lunghezza dati e parametri) sono sottoposte a controllo tramite `ZScoreService`. Le anomalie vengono segnalate e pubblicate come eventi, mentre le comunicazioni regolari vengono semplicemente registrate per fini diagnostici. Questo modulo fornisce una visione dettagliata delle operazioni PLC, consentendo di identificare comandi non usuali o accessi anomali a blocchi di dati (DB).

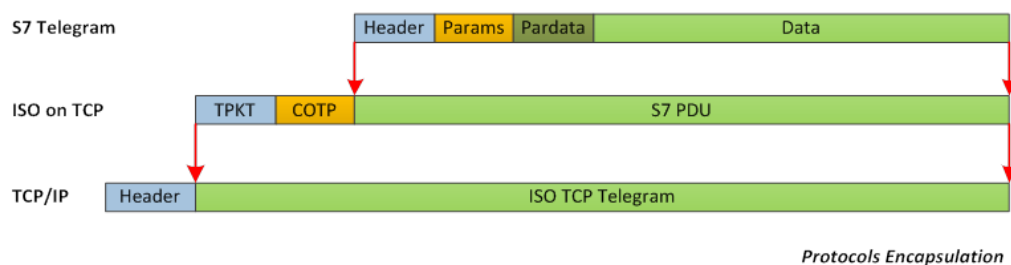


Figura 3.5: Struttura multilivello di un pacchetto Siemens S7Comm (TPKT → COTP → S7).

Handler OPC-UA Il modulo `opcua_packet_handler` analizza il traffico TCP (porta 4840) relativo al protocollo OPC-UA. Ogni pacchetto viene decodificato in

base ai campi del livello applicativo: `message_type` (HEL, ACK, OPN, MSG), `chunk_type`, `secure_channel_id`, `sequence_number`, `request_id` e payload binario. L'handler costruisce un oggetto evento contenente queste informazioni e applica un'analisi statistica sulle dimensioni e la sequenza dei pacchetti. Anche qui viene utilizzato `ZScoreService` per rilevare deviazioni dai profili normali di comunicazione, segnalando eventuali anomalie al sistema centrale. L'architettura è predisposta per integrare successivamente un modulo ML per la decodifica semantica del payload OPC-UA, consentendo in futuro una classificazione più profonda delle operazioni.

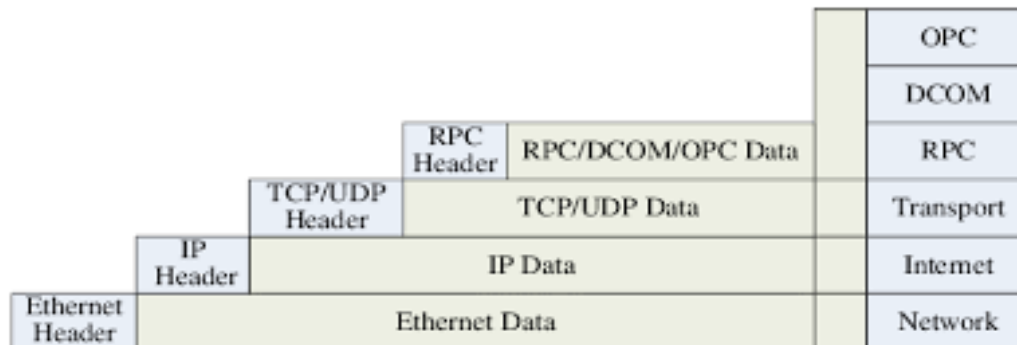


Figura 3.6: Struttura tipica di un pacchetto OPC-UA con header applicativo e campi di sessione.

Sintesi operativa Tutti gli handler condividono una struttura comune:

- verifica preliminare del livello di trasporto (TCP/UDP) e della porta di interesse;
- decodifica e parsing del payload applicativo tramite funzioni dedicate;
- estrazione di *feature* numeriche e simboliche per il motore di analisi;
- analisi statistica tramite Z-Score e, se necessario, validazione con modello ML;
- generazione di eventi strutturati e logging di sistema.

Questa architettura modulare permette di aggiungere nuovi protocolli SCADA con minimo impatto sul codice principale, mantenendo la compatibilità con il Dispatcher e con il sistema di rilevamento centralizzato.

3.2.4 Decodifica dei Payload Applicativi

Ciascun *handler* di protocollo integra un modulo di *decoding* dedicato per l'interpretazione dettagliata del payload applicativo, garantendo un'analisi coerente con le specifiche del protocollo industriale. La decodifica consente di estrarre campi semantici significativi (Function Code, Request ID, Data Block, ecc.) dai pacchetti

raw e di tradurli in strutture dati interpretabili dai moduli di analisi e rilevamento anomalie.

Parsing Modbus/TCP Il decoder per il protocollo Modbus/TCP effettua la suddivisione del payload in base alla struttura MBAP e al campo funzione. Ogni pacchetto è composto dai seguenti elementi principali:

- **Transaction ID** (2 byte): identificativo univoco della transazione;
- **Protocol ID** (2 byte): generalmente impostato a 0 per Modbus TCP;
- **Length** (2 byte): lunghezza del messaggio successivo;
- **Unit ID** (1 byte): indirizzo dello slave Modbus;
- **Function Code** (1 byte): operazione richiesta (lettura/scrittura);
- **Data** (variabile): contenuto del messaggio.

La funzione `parse_modbus_payload()` effettua l'unpacking del payload e gestisce sia le normali risposte di funzione, sia i messaggi di errore (exception codes). I codici funzione e le eccezioni sono mappati in dizionari interni per fornire una decodifica semantica leggibile, utile al motore di analisi statistico e ML. Un estratto del decoder è mostrato di seguito:

Listing 3.2: Funzione Parsing Modbus Payload

```
def parse_modbus_payload(payload: bytes) -> dict | None:
    """
    Parsing di un singolo pacchetto Modbus TCP (senza multi-
    ↳ frame)
    """
    if len(payload) < 8:
        return None % Payload troppo corto

    % MBAP Header: Transaction ID, Protocol ID, Length
    transaction_id, protocol_id, length = struct.unpack(">HHH"
    ↳ , payload[:6])
    unit_id = payload[6]
    function_code = payload[7]
    data = payload[8:] if len(payload) > 8 else b""

    parsed = {
        "transaction_id": transaction_id,
        "protocol_id": protocol_id,
        "length": length,
```



```

        "unit_id": unit_id,
        "function_code": function_code,
        "data": data,
    }

    % Gestione eccezioni Modbus
    if function_code >= 0x80:
        parsed["exception"] = True
        parsed["exception_code"] = data[0] if data else None
        parsed["exception_message"] = MODBUS_EXCEPTIONS.get(
            parsed["exception_code"], "Unknown_Exception"
        )
    else:
        parsed["exception"] = False
        parsed["function_name"] = MODBUS_FUNCTIONS.get(
            ↪ function_code, "Unknown"
        )

    return parsed

```

Parsing Siemens S7 (S7Comm) Il decoder S7 implementa una logica multilivello conforme alla pila TPKT → COTP → S7Comm. La funzione `parse_s7_payload()` effettua:

- (a) parsing dell'header TPKT (versione, lunghezza);
- (b) estrazione del PDU Type e dei parametri COTP;
- (c) interpretazione della PDU S7 con riferimento ai campi ROSCTR, Job Function e Data Block.

Ogni pacchetto viene arricchito con metadati semantici (funzione logica, area di memoria, riferimento DB, ecc.) per consentire un'analisi ad alto livello delle operazioni PLC. I dizionari `S7_JOB_FUNCTIONS`, `S7_MEMORY_AREAS` e `S7_ROSCTR` consentono di tradurre i codici numerici in descrizioni leggibili.

Listing 3.3: Funzione Parsing S7 Payload

```

def parse_s7_payload(payload: bytes) -> dict | None:
    """Parsing avanzato di pacchetto S7 TCP"""
    if len(payload) < 10:
        return None

    try:
        % --- TPKT Header ---

```

```
version, reserved, total_length = struct.unpack(">BBH"  
    ↪ , payload[:4])  
  
% --- COTP Header ---  
cotp_length = payload[4]  
cotp_pdu_type = payload[5]  
  
% --- S7 PDU Header ---  
s7_start = 4 + cotp_length  
if len(payload) < s7_start + 8:  
    return None  
  
pdu_type, rosctr, reserved1, pdu_ref, param_len,  
    ↪ data_len = struct.unpack(  
    ">BBHBBH", payload[s7_start : s7_start + 8]  
)  
  
% Parametri e dati  
param_data = payload[s7_start + 8 : s7_start + 8 +  
    ↪ param_len]  
actual_data = payload[  
    s7_start  
    + 8  
    + param_len : s7_start  
    + 8  
    + param_len  
    + min(data_len, len(payload) - (s7_start + 8 +  
    ↪ param_len))  
]  
  
parsed = {  
    "tpkt_version": version,  
    "tpkt_length": total_length,  
    "cotp_pdu_type": cotp_pdu_type,  
    "pdu_type": pdu_type,  
    "rosctr": rosctr,  
    "rosctr_name": S7_ROSCTR.get(rosctr, "Unknown"),  
    "pdu_reference": pdu_ref,  
    "parameter_length": param_len,  
    "data_length": data_len,  
    "parameters": param_data,
```

```

        "data": actual_data,
        "job_function": None,
        "job_function_name": None,
        "items": [],
    }

% Analisi Job Function
if param_data:
    function_code = param_data[0]
    parsed["job_function"] = function_code
    parsed["job_function_name"] = S7_JOB_FUNCTIONS.get
    ↪ (
        function_code, f"Unknown_{0x{function_code:02X}
    ↪ })"
    )

% Parsing Read/Write Items
items = []
if param_data and len(param_data) > 1 and param_data
    ↪ [0] in (0x04, 0x05, 0x2E):
    item_count = param_data[1]
    idx = 2
    for _ in range(item_count):
        if len(param_data) < idx + 4:
            break
        area_code = param_data[idx + 1]
        db_number = None
        start = None
        length = None
        % Area DB
        if area_code == 0x84 and len(param_data) >=
            ↪ idx + 8:
            db_number = struct.unpack(">H", param_data
                ↪ [idx + 2 : idx + 4])[0]
            start = struct.unpack(">H", param_data[idx
                ↪ + 4 : idx + 6])[0]
            length = struct.unpack(">H", param_data[
                ↪ idx + 6 : idx + 8])[0]
        items.append(
            {
                "area": area_code,

```

```

        "area_name": S7_MEMORY_AREAS.get(
            ↪ area_code, "Unknown"),
        "db_number": db_number,
        "start": start,
        "length": length,
    }
)
% avanzamento indice (semplificato)
idx += 8 if area_code == 0x84 else 4
parsed["items"] = items

return parsed
except Exception as e:
    print("Errore parsing S7:", e)
    return None

```

Parsing OPC-UA Il decoder OPC-UA gestisce la versione binaria del protocollo su TCP (porta 4840). La funzione `parse_opcua_payload()` analizza la sequenza dei campi:

- **Message Type** (3 char): tipo di messaggio, come HEL, ACK, OPN, MSG;
- **Chunk Type** (1 char): indica se il messaggio è completo o frammentato;
- **Secure Channel ID, Sequence Number, Request ID**;
- **Payload**: dati applicativi codificati in formato binario.

I campi principali vengono convertiti in dizionari serializzabili per il successivo invio al motore di rilevamento. Questo approccio consente di effettuare un'analisi comportamentale anche sui messaggi OPC-UA, valutando pattern di frequenza, sequenza e dimensione dei pacchetti.

Listing 3.4: Funzione Parsing OPC-UA Payload

```

def parse_opcua_payload(payload: bytes) -> dict | None:
    """
    Parsing minimale OPC-UA TCP Binary
    - TPKT Header: 4 bytes
    - UA Header: MessageType (3 char) + ChunkType (1 char)
    - Restituisce anche il payload in formato esadecimale JSON
    ↪ -serializzabile
    """

```

```
if len(payload) < 8: % TPKT + UA header minimo
    return None

% TPKT header: versione, reserved, length (big endian)
version = payload[0]
length = int.from_bytes(payload[2:4], byteorder="big")

% UA Binary message header
message_type = payload[4:7].decode(
    "ascii", errors="replace"
) % e.g., 'MSG', 'OPN', 'CLO'
chunk_type = payload[7:8].decode(
    "ascii", errors="replace"
) % 'F'=Final, 'A'=Intermediate

% Campi opzionali: SecureChannelId, SequenceNumber,
    ↳ RequestId
secure_channel_id = (
    int.from_bytes(payload[8:12], "little") if len(payload)
    ↳ ) >= 12 else None
)
sequence_number = (
    int.from_bytes(payload[12:16], "little") if len(
    ↳ payload) >= 16 else None
)
request_id = (
    int.from_bytes(payload[16:20], "little") if len(
    ↳ payload) >= 20 else None
)

% Payload dati veri e propri
raw_data = payload[20:] if len(payload) > 20 else b""

% Conversione in stringa esadecimale JSON-serializzabile
payload_data_serializable = raw_data.hex() if raw_data
    ↳ else None

return {
    "message_type": message_type,
    "chunk_type": chunk_type,
    "secure_channel_id": secure_channel_id,
```

```
"sequence_number": sequence_number ,
"request_id": request_id ,
"payload_data": payload_data_serializable ,
}
```

Architettura comune dei decoder Tutti i decoder condividono una struttura coerente:

- validazione preliminare della lunghezza del payload;
- estrazione dei campi principali mediante `struct.unpack()`;
- costruzione di un dizionario semantico con metadati leggibili;
- eventuale normalizzazione dei dati per l'analisi statistica e ML.

Questa uniformità semantica tra gli *handler* permette al sistema `side-sniff` di trattare protocolli eterogenei in modo unificato, garantendo compatibilità con i moduli di rilevamento e con il sistema di alerting MQTT.

3.2.5 Modulo di Anomaly Detection: Z-Score Service

Il modulo `ZScoreService` rappresenta il componente statistico centrale del sistema, impiegato da ciascun *Protocol Handler* per la valutazione dinamica delle anomalie nei flussi di pacchetti SCADA. La logica implementata si basa sull'analisi delle deviazioni standard (metrica Z-score) rispetto a valori di riferimento (*baseline*) specifici per ogni coppia `device-protocol`.

Architettura e Funzionamento

`ZScoreService` è strutturato come una *singleton class* thread-safe, garantendo l'accesso concorrente da parte di più thread di parsing provenienti dai vari *handler*. All'avvio, il modulo inizializza un insieme di baseline statistiche di default, definendo per ciascuna metrica di interesse — come `function_code` e `data_length` — una media (μ) e una deviazione standard (σ) di riferimento.

Ogni *Protocol Handler* (es. Modbus, S7, OPC-UA) invoca il metodo `check()` passando un dizionario di *features* estratte dal pacchetto corrente:

- **function_code**: operazione Modbus o tipo di servizio S7;
- **data_length**: dimensione del payload;
- **timing features**: (opzionali) tempo di risposta o frequenza di richieste.

Per ciascuna metrica numerica, il modulo calcola il valore:

$$Z = \frac{x - \mu}{\sigma}$$

dove x rappresenta il valore osservato nel pacchetto corrente. Se il valore assoluto $|Z|$ eccede la soglia configurata (`threshold`, di default 3), l'evento viene classificato come anomalo e accumulato in una coda temporale associata al dispositivo e protocollo di origine.

Meccanismo di Aggregazione Temporale

Per evitare falsi positivi dovuti a fluttuazioni isolate, il modulo impiega un meccanismo di aggregazione basato su una *deque* temporale (*sliding window*) di dimensione configurabile (`aggregation_window`). Solo quando il numero minimo di anomalie (`min_anomalies`) viene superato all'interno della finestra temporale, viene generato un evento di allarme consolidato, notificato al sistema superiore di alerting.

Aggiornamento Dinamico delle Baseline

Il modulo supporta un aggiornamento adattivo delle baseline, attivabile tramite il flag `update_dynamic=True`. In tale modalità, le statistiche vengono aggiornate in tempo reale secondo due strategie:

- (a) **Media Mobile Esponenziale (EMA)**: attivata con il parametro `use_ema=True`, utilizza un coefficiente α per dare maggiore peso agli eventi recenti:

$$\mu_t = \alpha x_t + (1 - \alpha)\mu_{t-1}$$

- (b) **Aggiornamento Lineare Medio**: in alternativa, viene calcolata una media aritmetica progressiva dei valori recenti per una stabilizzazione graduale.

Gestione della Concorrenza e Persistenza

L'intero accesso ai dati condivisi (statistiche, soglie, anomalie) è protetto da un `Lock` per garantire la coerenza in ambienti multithread, specialmente quando più *handler* aggiornano simultaneamente le stesse strutture dati. Le baseline e le soglie possono inoltre essere personalizzate per singolo dispositivo o protocollo attraverso i metodi:

- `set_baseline(device_id, protocol, baseline_stats)`
- `set_threshold(device_id, protocol, threshold)`

Integrazione con i Protocol Handler

Ciascun *handler* invoca `ZScoreService.getInstance().check()` durante la fase di analisi di ogni pacchetto decodificato. Se la funzione restituisce un'anomalia aggregata, il messaggio generato viene inviato al modulo di *alert management*, che può inoltrarlo via MQTT o loggarlo nel sistema SIEM. Questo approccio permette un'analisi statistica coerente e centralizzata, indipendentemente dal tipo di protocollo in uso.

Sintesi Operativa

Il flusso di funzionamento può essere riassunto come segue:

- (a) L'handler decodifica il pacchetto e ne estrae le feature numeriche;
- (b) Le feature vengono inviate al modulo `ZScoreService`;
- (c) Viene calcolato lo Z-score per ogni metrica;
- (d) Le anomalie vengono accumulate e analizzate in una finestra temporale;
- (e) Al superamento della soglia di aggregazione viene generato un evento di allarme di warning.

3.2.6 Modulo di Anomaly Detection: ML - Isolation Forest

Il modulo `IsolationForestService` rappresenta il componente di rilevazione anomalie basato su apprendimento automatico del sistema S.I.D.E., complementare al servizio statistico `ZScoreService`. La sua funzione è quella di individuare pattern di traffico anomali attraverso un modello di *Machine Learning* addestrato su flussi di pacchetti SCADA etichettati come “attacco” e “clean”, sfruttando la capacità dell'algoritmo `Isolation Forest` di isolare istanze rare o inconsuete nello spazio delle feature.

Architettura del Modulo e Dataset di Addestramento

Il processo di training del modello `IsolationForest` avviene in una fase offline, utilizzando un insieme di dataset CSV derivati da catture di traffico SCADA in due condizioni:

- **Dataset Clean:** rappresenta il comportamento legittimo dei dispositivi industriali;

- **Dataset Attack:** contiene diverse tipologie di attacchi (es. *Replay*, *Function Code Injection*, *Flooding*).

Durante il training, il sistema concatena i file CSV d'attacco, ne utilizza il 70Le feature di interesse, selezionate in funzione del protocollo (es. Modbus/TCP), comprendono:

Listing 3.5: Selezione delle Feature per ML-IsolationForestModbusService

```
features = ["pkt_size", "ip_ttl", "ip_len", "ip_proto_num", "  
    ↪ src_port", "dst_port", "modbus_function_code", "  
    ↪ modbus_data_length"]
```

Tali feature vengono pulite, normalizzate con uno `StandardScaler` e successivamente utilizzate per addestrare il modello di isolamento. Al termine, vengono generati e salvati su disco i seguenti artefatti:

- `isoforest_modbus_model.joblib`: modello addestrato;
- `isoforest_modbus_scaler.joblib`: scaler associato per la normalizzazione dei dati;
- `isoforest_modbus_threshold.txt`: soglia di decisione derivata dal training.

Meccanismo di Rilevazione

Durante l'analisi online, ciascun *Protocol Handler* (es. `ModbusHandler`) richiama il servizio `IsolationForestModbusService` tramite il metodo:

Listing 3.6: Utilizzo e integrazione del Modello di Machine Learning all'interno dell'Handler

```
IsolationForestModbusService.get_instance().check(  
    ↪ parsed_packet, raw_packet)
```

Il modulo opera come una *singleton class*, che carica all'avvio il modello, lo scaler e la soglia associati al protocollo in uso. Per ogni pacchetto, viene invocato un *feature extractor* che ricostruisce il vettore di input del modello a partire da:

- campi IP e TCP (`ttl`, `len`, `proto`, `sport`, `dport`);
- informazioni Modbus decodificate (`function_code`, `data_length`);
- dimensione complessiva del pacchetto.

Le feature vengono trasformate tramite lo `StandardScaler` e passate al modello per ottenere il punteggio di decisione (`decision_function`). Se il punteggio risulta inferiore alla soglia salvata (`threshold`), il pacchetto viene marcato come anomalo.

Threshold e Valutazione

La soglia di rilevamento è definita dinamicamente in fase di training come percentile dei punteggi di decisione osservati sui dati d'attacco, solitamente il 65° percentile. Questo consente un bilanciamento ottimale tra sensibilità e robustezza ai falsi positivi. Durante la validazione sul dataset “clean”, viene calcolata la percentuale di campioni erroneamente classificati come anomalie, utile per calibrare il valore di soglia finale.

Integrazione con il Sistema di Monitoraggio

Nel flusso operativo in tempo reale:

- (a) Il pacchetto viene decodificato dal *Protocol Handler*;
- (b) Le feature vengono estratte e scalate;
- (c) Il modello ML produce un punteggio e una classificazione binaria (*normal/anomaly*);
- (d) Se viene rilevata un'anomalia, l'evento è notificato al modulo di *alert management*, in modo analogo al servizio Z-Score.

Vantaggi e Considerazioni

L'approccio **Isolation Forest** consente di rilevare comportamenti anomali anche in assenza di regole esplicite o soglie statiche, risultando particolarmente efficace per:

- pattern complessi o non lineari nei flussi SCADA;
- deviazioni graduali dovute a compromissioni lente;
- reti multi-protocollo con dinamiche diverse tra dispositivi.

Rispetto al modulo **ZScoreService**, la componente ML fornisce un'analisi più profonda del contesto, ma richiede una fase di addestramento supervisionato e un'accurata selezione delle feature per ciascun protocollo industriale.

3.2.7 Flusso di Comunicazione: side-sniff e side-device-discovery

Il modulo **side-sniff** è responsabile della cattura dei pacchetti in tempo reale all'interno dell'ambiente ICS/SCADA. Una volta decodificati, i pacchetti vengono pubblicati in un canale di messaggistica intermedio, consumato dal servizio **side-device-discovery**.

Quest'ultimo agisce da *intermediario logico* tra il livello di cattura e il backend, svolgendo le seguenti funzioni:

- **Consumer:** riceve i messaggi dal `side-sniff`, contenenti informazioni grezze o pre-elaborate.
- **Publisher:** invia al modulo `side-api-back-end` eventi arricchiti con metadati e risultati statistici provenienti dallo `ZScoreService` o dai vari modelli di machine learning individuali.

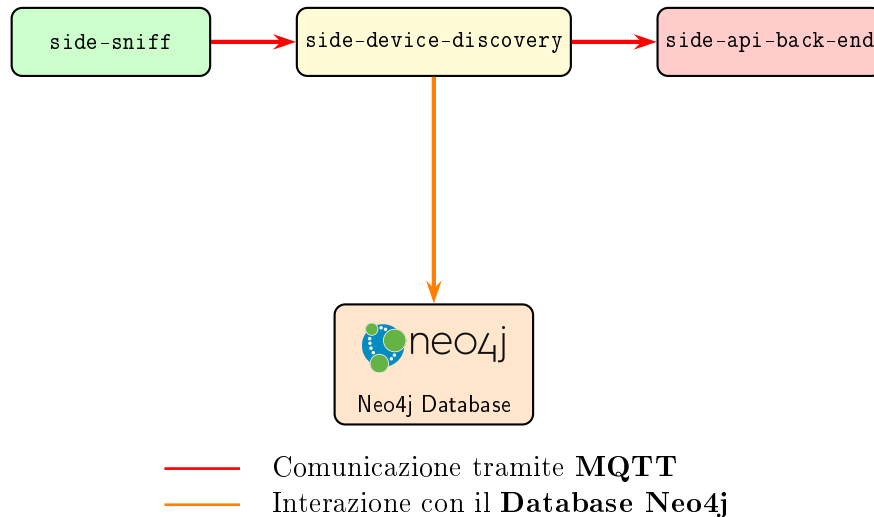


Figura 3.7: Flusso di comunicazione semplificato tra i moduli principali e il database.

3.3 side-device-discovery

Il modulo `side-device-discovery` è progettato come servizio in background, il cui scopo principale è quello di ricevere e gestire gli eventi pubblicati dal modulo `side-sniff`, consentendo successivamente la loro distribuzione verso i moduli di backend e front-end. In questo modo, `side-device-discovery` funge sia da **consumer** che da **publisher** nel flusso di comunicazione del sistema.

3.3.1 Funzionamento generale

Il modulo si comporta come un servizio persistente che esegue le seguenti operazioni:

1. Si collega a un broker RabbitMQ per consumare gli eventi pubblicati dal modulo `side-sniff`.
2. Processa ciascun evento tramite handler registrati dinamicamente, distinguendo i tipi di evento (ad esempio, `new_device` o `device_update`).
3. Inserisce o aggiorna le informazioni sui dispositivi all'interno del database Neo4j, mantenendo la topologia dei dispositivi e le relazioni di comunicazione.

4. Pubblica gli eventi elaborati su un exchange dedicato verso i moduli `side-api-back-end` e `side-front-end`.

3.3.2 Struttura del codice

Il modulo è composto principalmente dai seguenti file e componenti:

1. `side-device-discovery.py` Questo script rappresenta il cuore del modulo. Le sue funzioni principali sono:

- **Registrazione degli handler:** tramite il decorator `register_handler` ogni funzione viene associata a una `routing_key`, consentendo la gestione dinamica degli eventi ricevuti.
- **Callback dei messaggi:** ogni messaggio ricevuto da RabbitMQ viene elaborato tramite la funzione `callback`, che richiama l'handler corrispondente e passa il driver Neo4j per la persistenza.
- **Inizializzazione del servizio:** il modulo si collega a RabbitMQ, dichiara l'exchange `discovery.events` e crea una coda temporanea per ricevere tutti i messaggi dei routing key registrati.
- **Gestione terminazione:** viene gestita la chiusura sicura di connessioni al broker e al database in caso di interruzione.

2. `services/database_service.py` Contiene la funzione `register_modbus_event` che gestisce l'upsert dei nodi e delle relazioni all'interno del database Neo4j. In particolare:

- Crea o aggiorna i nodi dei dispositivi (`NODE_DEVICE`) identificati da IP e porta.
- Definisce le relazioni bidirezionali (`RELATION_COMMUNICATES_WITH`) tra dispositivi, memorizzando informazioni come protocollo, timestamp e conteggio degli eventi.
- Supporta la persistenza di dati Modbus, ma può essere esteso ad altri protocolli.

3. `configuration/device_discovery_publisher.py` Implementa il servizio di pubblicazione verso RabbitMQ:

- Inizializza una connessione al broker e dichiara un `exchange` di tipo `fanout`.
- Permette la pubblicazione di eventi elaborati verso tutti i subscriber collegati.
- Gestisce la chiusura sicura della connessione.

3.3.3 Flusso operativo

Il flusso operativo del modulo può essere sintetizzato come segue:

1. `side-sniff` pubblica un messaggio contenente le informazioni dei pacchetti catturati.
2. `side-device-discovery` riceve il messaggio tramite RabbitMQ e lo passa al handler registrato.
3. L'handler utilizza il driver Neo4j per aggiornare la rappresentazione dei dispositivi e delle loro relazioni.
4. Il messaggio elaborato viene pubblicato sull'exchange `frontend.events`, rendendolo disponibile per i moduli di backend e frontend.

3.3.4 Proprietà principali

- Funziona come servizio in background persistente.
- Gestisce eventi di tipo `new_device` e `device_update`.
- Interagisce direttamente con il database Neo4j per garantire la consistenza dei dati dei dispositivi.
- Pubblica eventi verso sistemi downstream tramite RabbitMQ, implementando un pattern *publish-subscribe*.

3.4 side-api-backend

Il modulo `side-api-back-end` costituisce il componente di interfaccia tra il *core system* e il *front-end* dell'ambiente S.I.D.E. (SCADA Intrusion Detection Environment). È implementato come server `FastAPI` eseguibile tramite `Uvicorn`, e integra in un unico livello applicativo:

1. la gestione delle connessioni WebSocket per la comunicazione in tempo reale con l'interfaccia utente;
2. l'esposizione di API REST per interrogazioni dirette (es. elenco dispositivi, relazioni, metriche);
3. la gestione della persistenza e della topologia di rete su database `Neo4j`;
4. l'integrazione asincrona con il broker `RabbitMQ` per la ricezione di eventi generati dai moduli di scoperta e analisi.

3.4.1 Struttura generale

La struttura principale del modulo è organizzata nei seguenti componenti:

- **run.py**: punto d'ingresso del server applicativo;
- **configuration/**: contiene le impostazioni di sistema e i moduli di inizializzazione (database, WebSocket);
- **services/**: implementa la logica di business (query Neo4j, formattazione dati);
- **controllers/**: definisce gli endpoint REST e WebSocket esposti al front-end;
- **service_discovery_consumer.py**: gestisce un processo di ascolto RabbitMQ per l'inoltro di eventi in tempo reale al front-end.
- **service_discovery_alert_consumer.py**: gestisce un processo di ascolto RabbitMQ per l'inoltro di eventi di alert in tempo reale al front-end.
- **websocket_manager.py**: gestisce due websockets per l'inoltro dei messaggi arrivati dai canali di comunicazione di RabbitMQ

3.4.2 Punto di avvio: run.py

Il file **run.py** costituisce il *bootstrap* del server. Esso costruisce dinamicamente il riferimento all'applicazione **FastAPI** utilizzando i parametri definiti nel file di configurazione:

Listing 3.7: Setting up iniziale del server fastapi

```
app_location = f"{MODULE_NAME}:{APP_NAME}"

uvicorn.run(app_location, host=HOST, port=PORT, reload=RELOAD)
```

Il server è lanciato tramite **Uvicorn**, un web server asincrono compatibile con ASGI (*Asynchronous Server Gateway Interface*), che consente una gestione concorrente efficiente delle richieste.

3.4.3 Gestione del database Neo4j

Il modulo **configuration/database_configuration.py** implementa la connessione al database a grafo Neo4j mediante il driver ufficiale **neo4j-driver**. L'inizializzazione del driver è effettuata tramite la funzione:

Listing 3.8: Dichiarazione dei driver NEO4J in Python

```
driver = GraphDatabase.driver(URI, auth=(USER, PASSWORD))
```

I parametri di connessione vengono caricati da file `.env` per garantire la separazione tra codice e configurazione. La classe `SideDatabaseConfiguration` incapsula i parametri di connessione e offre metodi getter/setter validati. Il modulo definisce inoltre le entità principali (`Device`) e le relazioni (`COMMUNICATES_WITH`) utilizzate per modellare le topologie ICS/SCADA, includendo proprietà quali:

- `ip`, `port`, `role`, `protocol`, `first_seen`, `last_seen` per i nodi `Device`;
- `protocol`, `total_queries`, `error_count` e `queries_by_function` per le relazioni di comunicazione.

3.4.4 Gestione degli eventi asincroni: `service_discovery_consumer.py`

Il file `service_discovery_consumer.py` avvia un RabbitMQ consumer per ricevere notifiche e aggiornamenti di rete provenienti dal modulo `ServiceDiscovery` o da altri microservizi. Utilizza la libreria `pika` per instaurare una connessione al broker, sottoscrivendosi all'exchange:

Listing 3.9: Dichiarazione dell'exchange RabbitMQ in Python

```
exchange_name = "frontend.events"  
channel.exchange_declare(exchange=exchange_name, exchange_type="  
    ↪ fanout")
```

Ogni messaggio ricevuto viene deserializzato (formato JSON) e inoltrato in modo asincrono al gestore WebSocket `ws_manager.broadcast(event)`, garantendo un flusso in tempo reale verso i client front-end connessi.

3.4.5 Gestione connessioni WebSocket: `websocket_manager.py`

Il componente `WebSocketManager` fornisce una gestione centralizzata delle connessioni WebSocket attive. È responsabile della ricezione e dell'invio dei messaggi tra il back-end e i client:

- `connect(ws)`: accetta una nuova connessione;
- `disconnect(ws)`: rimuove connessioni interrotte;
- `broadcast(message)`: invia in broadcast messaggi JSON a tutti i client attivi.

Il manager mantiene le connessioni attive in un set tipizzato `Set[WebSocket]`, assicurando la scalabilità anche in presenza di più sessioni contemporanee.

3.4.6 Controller REST e WebSocket

Due controller principali orchestrano la comunicazione:

1. `graph_controller.py` — espone endpoint REST come `/api/devices`, che interroga Neo4j tramite il servizio `get_devices(driver)` e restituisce la topologia o i dispositivi attivi;
2. `websocket_controller.py` — gestisce il canale `/ws/network`, accettando connessioni bidirezionali e mantenendole attive per lo streaming continuo di dati (eventi, metriche, anomalie).

3.4.7 Integrazione dei servizi e funzionamento complessivo

Il modulo `side-api-back-end` funge da *orchestratore asincrono*, integrando servizi eterogenei:

- riceve in tempo reale eventi e notifiche di scoperta rete via `RabbitMQ`;
- li inoltra istantaneamente ai client tramite `WebSocketManager`;
- fornisce API REST per consultare lo stato della rete, dispositivi e metriche analitiche;
- interagisce con il database `Neo4j` per aggiornare nodi e relazioni in base agli eventi ricevuti.

Questo consente al front-end di ottenere una rappresentazione dinamica della rete ICS monitorata, in cui dispositivi, connessioni e allarmi vengono aggiornati in tempo reale senza necessità di polling continuo.

3.4.8 Sintesi del flusso operativo

1. Il modulo viene avviato tramite `python run.py`;
2. `Uvicorn` avvia l'applicazione `FastAPI`;
3. Il consumer `RabbitMQ` ascolta sull'exchange `frontend.events`;
4. Gli eventi ricevuti vengono trasmessi ai client via `WebSocket`;
5. Le API REST permettono al front-end di interrogare lo stato della rete e le entità `Neo4j`.

3.4.9 Ruolo architetturale

Il modulo `side-api-backend` rappresenta il *nodo di interfaccia logico* tra il sistema di rilevamento (livello back-end) e il sistema di visualizzazione (front-end). Funge da punto unico di ingresso per tutti i dati, trasformando eventi di basso livello (pacchetti, relazioni, metriche) in strutture semantiche comprensibili al front-end, mantenendo un modello coerente della rete e dei flussi di comunicazione industriali.

3.4.10 Flusso di Interazione con il Backend

Il modulo `side-api-back-end` riceve gli eventi già filtrati ed elaborati. A questo livello vengono aggregati i risultati provenienti dai vari protocolli e memorizzati nel database centrale. Ogni evento include le informazioni statistiche fornite dal `ZScoreService`, integrato direttamente nel modulo `side-sniff`, consentendo una visione globale dello stato dei dispositivi.

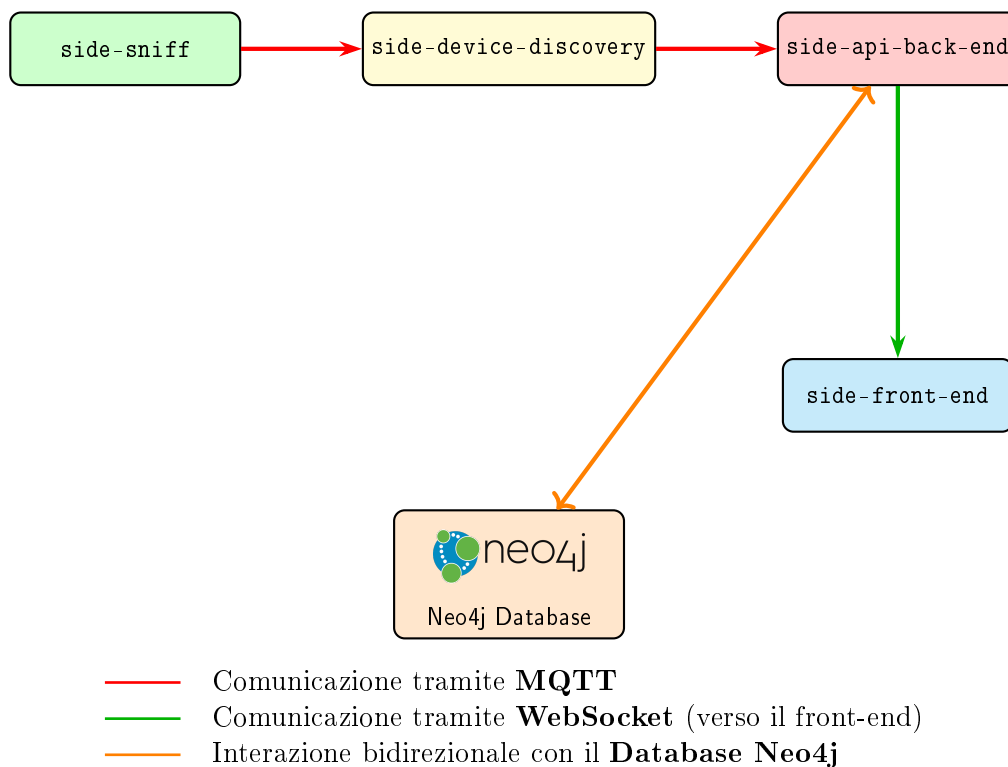


Figura 3.8: Flusso semplificato tra i moduli principali del sistema, con interazione bidirezionale tra `side-device-discovery` e il database Neo4j.

3.5 side-frontend

Il modulo *frontend* è sviluppato in **Next.js** e rappresenta l'interfaccia grafica principale del sistema **S.I.D.E. (SCADA Intrusion Detection Environment)**. Il suo obiettivo

è fornire una visualizzazione interattiva e in tempo reale della rete SCADA, consentendo all'operatore di monitorare in modo immediato lo stato dei dispositivi, le comunicazioni attive e gli eventi di sicurezza rilevati.

3.5.1 Dashboard interattiva

Appena effettuato l'accesso, l'operatore viene accolto da una **dashboard dinamica** che mostra la topologia della rete SCADA. Attraverso una connessione **WebSocket** con il modulo *side-api-backend*, il frontend riceve continuamente aggiornamenti riguardanti lo stato dei dispositivi e li rappresenta graficamente in un **grafo 3D interattivo**. In questa vista è possibile osservare in tempo reale le interazioni tra *client/server* (o *master/slave*), con i nodi che rappresentano i dispositivi e gli archi che indicano le comunicazioni attive. Eventuali anomalie o stati di allerta vengono evidenziati tramite codici colore o icone distintive, facilitando un'immediata comprensione della situazione di rete.

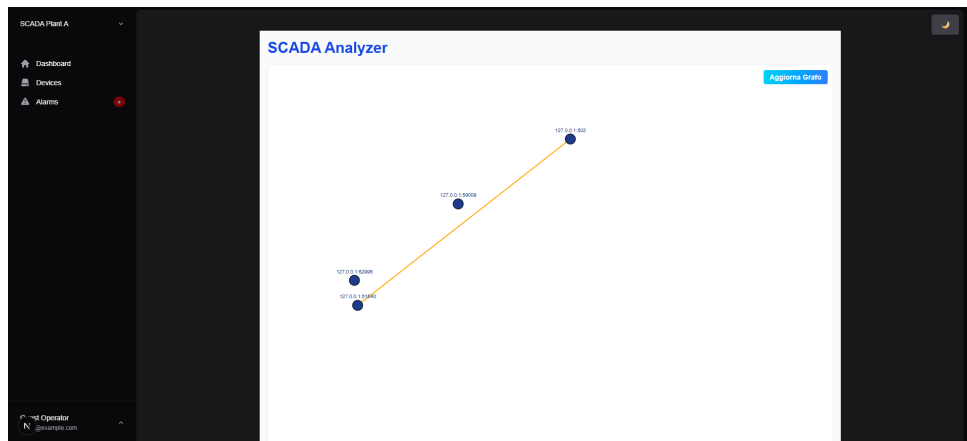


Figura 3.9: Dashboard 3D interattiva della rete SCADA: i nodi rappresentano i dispositivi, mentre gli archi mostrano le comunicazioni attive.

3.5.2 Sezione /devices

All'interno della sezione */devices*, l'utente può consultare l'elenco completo dei dispositivi monitorati. I device vengono automaticamente categorizzati in base al protocollo industriale rilevato (**Modbus**, **OPC-UA**, **S7**) e possono essere filtrati dinamicamente. Ogni dispositivo è rappresentato tramite una **card informativa** che mostra:

- indirizzo IP e porta di comunicazione;
- ruolo nel sistema (*client/server*);
- stato corrente (*attivo* o *inattivo*);
- protocollo di riferimento.

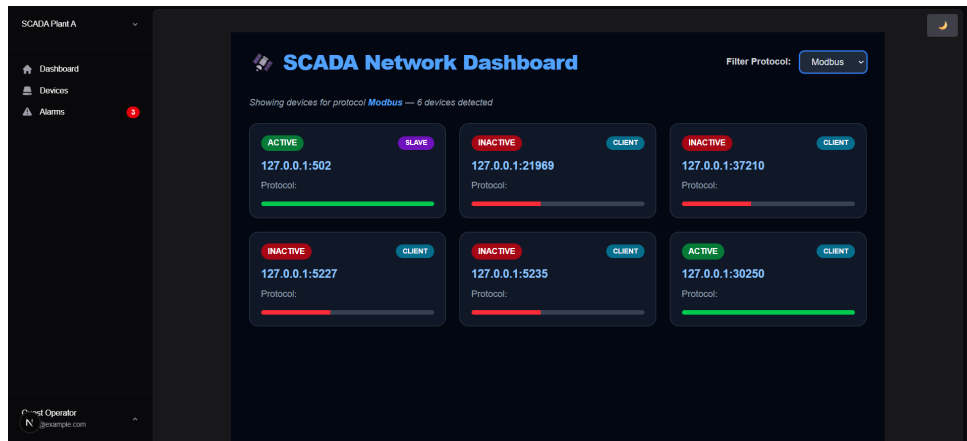


Figura 3.10: Visualizzazione dei dispositivi SCADA e relativo filtraggio per protocollo.

3.5.3 Sezione /events

La sezione dedicata agli *eventi* o *allarmi* consente di analizzare in dettaglio le segnalazioni di sicurezza. Tramite un sistema di filtraggio basato sulla **severity** dell'evento (ad esempio `EARLY_WARNING_ZSCORE` o `CONFIRMED_ANOMALY_ML`), l'operatore può isolare rapidamente gli episodi di interesse. Ogni evento può essere espanso in una card dedicata che mostra:

- il tipo di anomalia rilevata;
- il dispositivo coinvolto e il protocollo di riferimento;
- la descrizione dettagliata del pacchetto o dei dati analizzati.

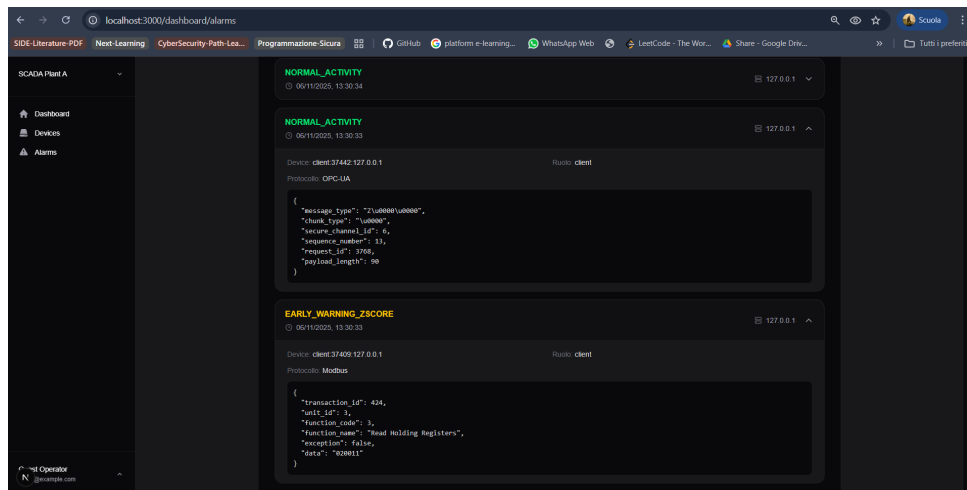


Figura 3.11: Visualizzazione dei vari eventi attraverso livelli di Severity

Questo consente una comprensione approfondita del contesto operativo e del comportamento anomalo individuato dal sistema di rilevazione.

Un altro componente grafico molto intuitivo è quello di **Neo4J**, che supporta la visualizzazione dei dati memorizzati nel database a grafo. Nel grafo sono rappresentati i seguenti nodi principali:

- **Device** — rappresenta un dispositivo presente nella rete SCADA;
- **Anomaly** — indica un evento anomalo rilevato dai moduli di analisi (Z-Score o Machine Learning).

Le relazioni tra i nodi sono invece:

- **COMMUNICATES_WITH** — indica le comunicazioni attive tra due dispositivi;
- **HAS_EVENTS** — collega un dispositivo agli eventi o alle anomalie rilevate durante l'interazione.

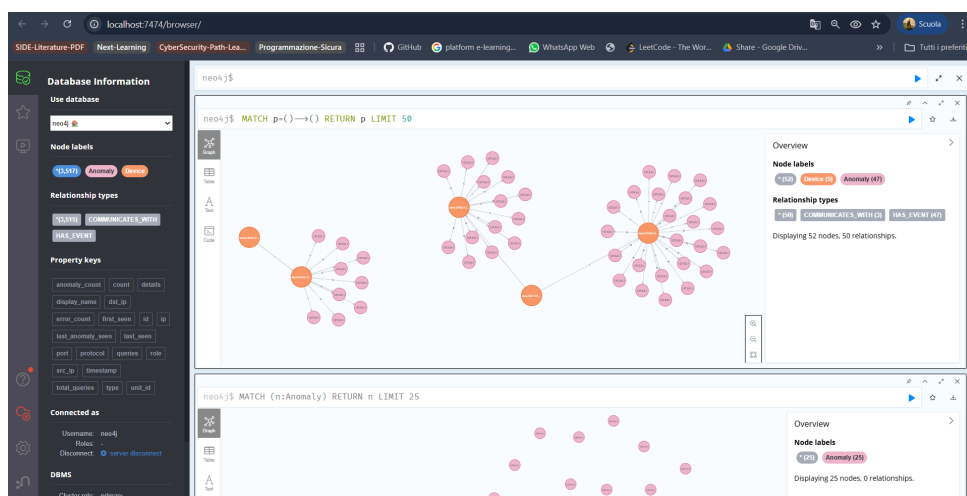


Figura 3.12: Visualizzazione del grafo dei dispositivi e delle anomalie in Neo4J.

Questa rappresentazione consente di comprendere rapidamente la frequenza e la distribuzione degli eventi nella rete, distinguendo tra anomalie leggere (rilevate tramite *Z-Score*) e anomalie gravi (individuate tramite modelli di *Machine Learning*, come l'*Isolation Forest*).

3.5.4 Avvio del modulo

Per avviare il modulo *frontend*, è sufficiente installare la versione LTS v22.16.0 di Node.js, accedere alla directory del progetto ed eseguire il comando:

```
npm run dev
```

3.6 side-ml-models

All'interno del modulo `side-ml-models`, il processo di addestramento del modello di rilevazione anomalie si articola in due fasi principali: estrazione dei dati dai file PCAP e training del modello *Isolation Forest*.

3.6.1 Estrazione dei dati dai file PCAP

I pacchetti di rete catturati in contesti SCADA/ICS sono organizzati nella cartella `data-modbus`. L'elaborazione dei pacchetti avviene mediante lo script `pcap_to_csv_enhanced.py`, che realizza le seguenti operazioni:

1. **Scansione dei file PCAP:** tutti i file nella directory `data-modbus` vengono letti e processati in sequenza.
2. **Parsing dei protocolli:** identificazione di ARP, IP, TCP, UDP e ICMP, con estrazione delle principali feature:
 - `src_ip`, `dst_ip`
 - `src_port`, `dst_port`
 - `pkt_size`, `ip_ttl`, `ip_proto_num`
3. **Parsing Modbus TCP:** per i pacchetti con porta TCP 502, estrazione di:
 - `function_code`, `data_length`
 - eventuali eccezioni (`exception_code`, `exception_message`)
4. **Creazione del dataset:** i dati estratti vengono convertiti in CSV tramite un DataFrame Pandas, includendo un file di riepilogo con statistiche sulle feature principali.

3.6.2 Addestramento del modello Isolation Forest

Il dataset CSV ottenuto viene utilizzato per addestrare un modello `Isolation Forest`, con le seguenti fasi:

1. **Selezione dei dati di training:** dal dataset di attacco, il 70% dei record viene scelto per l'addestramento; il restante 30% può essere utilizzato per validazione.
2. **Preprocessing:** le feature numeriche vengono standardizzate mediante `StandardScaler` fit solo sui dati d'attacco.
3. **Addestramento:** il modello `Isolation Forest` viene addestrato sui dati standardizzati, con parametri configurabili (`n_estimators`, `contamination`).
4. **Determinazione della soglia:** la soglia di decisione viene calcolata come percentile specifico dei punteggi di training (`decision_function`), utile per valutare anomalie tramite Z-Score.

5. **Valutazione:** test del modello sul sottoinsieme di attacchi residui e sul dataset clean, calcolando la percentuale di falsi positivi e visualizzando la distribuzione dei punteggi.
6. **Salvataggio degli artefatti:** modello, scaler e soglia vengono serializzati e salvati su disco in formato `.joblib`, permettendo l'integrazione all'interno dei diversi Handler di rilevazione anomalie.

3.6.3 Osservazioni

Ogni Handler dispone di un modello singolo e univoco, addestrato su dataset differenti selezionati in base alla tipologia di protocollo (ad esempio **Modbus**, **OPC-UA**, **S7**). Grazie alla serializzazione in `.joblib`, ogni Handler può caricare il proprio modello in fase di esecuzione e valutare in tempo reale se un evento supera la soglia di Z-Score, permettendo l'attivazione di allarmi o trigger specifici.

Questa struttura garantisce:

- la modularità dei modelli per protocollo, evitando conflitti tra diverse tipologie di traffico;
- la possibilità di aggiornare o riaddestrare singoli modelli senza influenzare gli altri;
- l'integrazione fluida nella pipeline di rilevazione anomalie del sistema SCADA/ICS.

3.7 Device Simulators

Per testare l'ecosistema SCADA/ICS, sono stati implementati simulatori per diversi protocolli industriali: **Modbus TCP**, **Siemens S7** e **OPC-UA**. Ogni simulatore comprende un server (che espone registri/variabili/metodi) e un client (che genera traffico e simula dispositivi reali).

3.7.1 Modbus TCP

Server Modbus TCP

Il server Modbus TCP è basato su `pymodbus==3.0.2` e implementato in modalità asincrona (`asyncio`). Gestisce più `unit_id` simultaneamente tramite `ModbusServerContext`, ognuno con un `ModbusSlaveContext` contenente:

- **Discrete Inputs (DI):** inizializzati a 15;
- **Coils (CO):** inizializzati a 16;

- **Holding Registers (HR)**: inizializzati a 17;
- **Input Registers (IR)**: inizializzati a 18.

Il server espone informazioni identificative standard tramite `ModbusDeviceIdentification` e fornisce un logging a livello `DEBUG` per tracciare tutte le richieste. L'avvio avviene tramite la funzione `run_modbus_server()`.

Client Modbus TCP

Il client genera traffico verso il server simulato. Per ogni `unit_id`:

- Scrive coil casuali con possibilità di errori simulati;
- Legge coil e holding register;
- Introduce ritardi e pause casuali per simulare condizioni di rete variabili.

Tutte le operazioni gestiscono eccezioni (`ModbusException`) per garantire stabilità e realismo. L'obiettivo principale è testare il comportamento del server e del frontend SCADA in scenari sia normali che anomali.

3.7.2 Siemens S7 (Snap7)

Server S7

Il server S7 è simulato tramite `snap7` e ascolta sulla porta TCP 102. Permette la gestione di:

- *Data Blocks (DB)*;
- Merkers (MK), Timer (TM) e Contatori (CT).

Il server replica il comportamento di un PLC Siemens base, consentendo test realistici senza dispositivi fisici.

Client S7

Il client S7 avanzato (`S7AdvancedClient`) consente:

- Connessione al server configurabile con IP, rack, slot e porta;
- Lettura e scrittura nei DB e nelle aree di memoria (MK, TM, CT);
- Gestione dei principali tipi dati PLC: interi, real, booleani e stringhe;

- Loop di simulazione che aggiorna valori incrementali, toggle di bit e scrittura di stringhe.

Il logging permette di tracciare connessioni, operazioni e errori, mentre il ciclo continuo fornisce dati dinamici per il frontend SCADA e per test di monitoraggio o controllo remoto.

3.7.3 OPC-UA

Server OPC-UA

Il server OPC-UA, implementato con `asyncua`, espone un *address space* con oggetti e variabili scrivibili.

Configurazione

- Endpoint TCP: `opc.tcp://0.0.0.0:4840/freeopcua/server/`;
- Namespace personalizzato: `http://examples.freeopcua.github.io`;
- Oggetto principale: `MyObject`.

Variabili simulate

- **Temperature:** 20–100;
- **Pressure:** 0–10;
- **Level:** 0–500.

Aggiornate periodicamente da un task asincrono con variazioni casuali per simulare fenomeni reali.

Metodi server Metodi esposti tramite decoratore `@uamethod`, ad esempio `ServerMethod` per raddoppiare valori, e possibilità di definire metodi custom come `ResetCounter`.

Client OPC-UA

Il client OPC-UA comunica con il server e consente:

- Lettura e scrittura variabili con variazioni casuali;
- Invocazione di metodi server con argomenti casuali;
- Logging completo di tutte le operazioni.

Il loop asincrono permette di simulare scenari realistici, generando dati dinamici per il frontend SCADA e validando le logiche di monitoraggio e controllo.

3.7.4 Conclusioni sui simulatori

L'integrazione dei tre protocolli permette di costruire un ambiente SCADA/ICS completamente simulato:

- Testare la comunicazione client-server per diversi protocolli industriali;
- Generare traffico realistico per il frontend SCADA;
- Simulare anomalie, ritardi e variazioni dinamiche senza dispositivi fisici;
- Validare scenari di monitoraggio, controllo remoto e sicurezza.

4. SCADA – Cyber Attacks

4.1 Introduzione

In questo capitolo descriviamo gli scenari d'attacco riprodotti nel testbed, utilizzando come riferimento un dataset Modbus/TCP (baseline). Le sperimentazioni sono eseguite esclusivamente in un ambiente di laboratorio isolato e con autorizzazione esplicita; i risultati sono confrontati con studi e dataset pubblici nel dominio SCADA/ICS [14, 9].

4.2 Metodo di riproduzione del traffico

Per generare le condizioni sperimentali abbiamo adottato due modalità principali:

1. **Replay offline:** riproduzione dei pcap originali tramite strumenti come `tcpreplay` (utile per valutare l'impatto temporale a livello di rete).
2. **Rigenerazione programmata:** creazione/alterazione di flussi Modbus con script (ad es. `Scapy`) per ottenere varianti manipulate, testare payload modificati o pattern temporali specifici.

Con queste modalità abbiamo costruito due condizioni sperimentali:

Baseline traffico originale tratto dal dataset.

Manipolato traffico derivato dal baseline e opportunamente alterato per simulare gli attacchi descritti di seguito.

4.3 Obiettivi sperimentali

Gli esperimenti mirano a:

- valutare l'impatto funzionale sugli asset ICS (errori Modbus, eccezioni, variazione di valori dei registri);
- misurare l'efficacia di rilevamento del sistema `side-sniff` (IDS) nel riconoscere anomalie o attacchi;
- fornire materiale riproducibile per analisi comparate in laboratorio controllato.

4.4 Tipologie d'attacco considerate

Di seguito vengono descritte in modo sintetico e tecnico le tipologie d'attacco simulate nel testbed.

4.4.1 Man-In-The-Middle (MITM)

Un nodo intermedio intercetta e potenzialmente modifica richieste e risposte Modbus/TCP tra master e slave. Obiettivi tipici:

- lettura non autorizzata dei payload Modbus;
- iniettare risposte false (modifica dei register values);
- introdurre latenza o delay per studiare l'effetto sulla logica applicativa.

Nel nostro setup la manipolazione è effettuata offline sui pcap o durante la rigenerazione tramite script per evitare operazioni su sistemi reali.

4.4.2 modbusQuery2Flooding

Variante mirata a inviare coppie di query Modbus ripetute in rapida successione verso lo stesso o più slave. Scopo: saturare risorse locali (buffer, CPU, thread) e provocare degrado o malfunzionamenti del servizio Modbus.

4.4.3 modbusQueryFlooding

Flooding generico di query Modbus (letture o scritture) ad alta frequenza; può includere più function code per rendere il pattern meno prevedibile e più difficile da bloccare con semplici rate-limit.

4.4.4 pingFloodDDoS (ICMP)

Attacco ICMP Echo Request in elevato volume verso l'infrastruttura target. Effetti tipici:

- degrado della raggiungibilità della rete di controllo;
- aumento dei tempi di risposta ed eventuale perdita di pacchetti.

4.4.5 tcpSYNFloodDDoS

Generazione massiva di pacchetti TCP SYN senza completare la handshake, con l'obiettivo di esaurire il backlog di connessioni del target e impedire nuove connessioni legittime (ad esempio verso il servizio Modbus/TCP sulla porta 502).

4.5 Organizzazione dei test e note pratiche

- Per ogni attacco si definisce una serie di parametri controllati (tasso di invio, durata, target IP/porta, payload modificati) e si esegue almeno una prova di baseline per confronto.
- Tutti i pcap e gli script usati per la rigenerazione/alterazione sono conservati e hashati (SHA256) per garantire riproducibilità e tracciabilità.
- Le misure raccolte includono: latenza request–response, errori/exception Modbus, eventuali cambi di stato nei dispositivi simulati e log/alert prodotti da `side-sniff`.

5. Conclusioni

Il progetto S.I.D.E. ha dimostrato l'efficacia di un ambiente simulato per sistemi ICS/-SCADA, capace di generare traffico realistico e testare la resilienza dei protocolli Modbus TCP, Siemens S7 e OPC-UA. L'implementazione di server e client simulati ha permesso di osservare in tempo reale il comportamento dei sistemi e degli IDS, con gestione di anomalie controllate e monitoraggio accurato.

Sviluppi futuri Tra le principali direzioni di evoluzione del progetto si segnalano:

- **Automazione CI/CD:** creazione di ambienti simulati containerizzati (Docker) con configurazioni di rete variabili per testare singoli protocolli per volta in vari scenari SCADA in pipeline automatizzate;
- **Attacchi controllati dal front-end:** possibilità di generare scenari offensivi direttamente dall'interfaccia utente per valutare l'efficacia delle difese e il comportamento degli IDS;
- **Machine Learning avanzato:** integrazione di modelli sofisticati e dataset estesi sui protocolli industriali per migliorare la rilevazione delle anomalie e la previsione di attacchi complessi. Ad esempio : GNN (Graph Neural Network)

Bibliografia

- [1] Shahrulniza Musa, AAmir Shahzad e Abdulaziz Aborujilah. «Simulation base implementation for placement of security services in real time environment». In: *Proceedings of the 7th International Conference on Ubiquitous Information Management and Communication*. ICUIMC '13. Kota Kinabalu, Malaysia: Association for Computing Machinery, 2013. ISBN: 9781450319584. DOI: [10.1145/2448556.2448587](https://doi.org/10.1145/2448556.2448587). URL: <https://doi.org/10.1145/2448556.2448587>.
- [2] Y. Yang et al. «Rule-based intrusion detection system for SCADA networks». In: *2nd IET Renewable Power Generation Conference (RPG 2013)*. 2013, pp. 1–4. DOI: [10.1049/cp.2013.1729](https://doi.org/10.1049/cp.2013.1729).
- [3] Hui Lin et al. «Semantic Security Analysis of SCADA Networks to Detect Malicious Control Commands in Power Grids (Poster)». In: *Proceedings of the 7th International Conference on Security of Information and Networks*. SIN '14. Glasgow, Scotland, UK: Association for Computing Machinery, 2014, pp. 492–495. ISBN: 9781450330336. DOI: [10.1145/2659651.2659746](https://doi.org/10.1145/2659651.2659746). URL: <https://doi.org/10.1145/2659651.2659746>.
- [4] Shahir Majed, Suhaimi Ibrahim e Mohamed Shaaban. «Architecting and Development of the SecureCyber: A SCADA Security platform Over Energy Smart Grid». In: *Proceedings of the 16th International Conference on Information Integration and Web-Based Applications & Services*. iiWAS '14. Hanoi, Viet Nam: Association for Computing Machinery, 2014, pp. 170–174. ISBN: 9781450330015. DOI: [10.1145/2684200.2684308](https://doi.org/10.1145/2684200.2684308). URL: <https://doi.org/10.1145/2684200.2684308>.
- [5] Rishabh Samdarshi, Nidul Sinha e Paritosh Tripathi. «A triple layer intrusion detection system for SCADA security of electric utility». In: *2015 Annual IEEE India Conference (INDICON)*. 2015, pp. 1–5. DOI: [10.1109/INDICON.2015.7443439](https://doi.org/10.1109/INDICON.2015.7443439).
- [6] Yosra Lakhthar, Slim Rekhis e Nouredine Boudriga. «An Approach To A Graph-Based Active Cyber Defense Model». In: *Proceedings of the 14th International Conference on Advances in Mobile Computing and Multi Media*. MoMM '16. Singapore, Singapore: Association for Computing Machinery, 2016, pp. 261–268. ISBN: 9781450348065. DOI: [10.1145/3007120.3007142](https://doi.org/10.1145/3007120.3007142). URL: <https://doi.org/10.1145/3007120.3007142>.

- [7] Jeyasingam Nivethan e Mauricio Papa. «A SCADA Intrusion Detection Framework that Incorporates Process Semantics». In: *Proceedings of the 11th Annual Cyber and Information Security Research Conference*. CISRC '16. Oak Ridge, TN, USA: Association for Computing Machinery, 2016. ISBN: 9781450337526. DOI: [10.1145/2897795.2897814](https://doi.org/10.1145/2897795.2897814). URL: <https://doi.org/10.1145/2897795.2897814>.
- [8] Kevin Wong et al. «Enhancing Suricata intrusion detection system for cyber security in SCADA networks». In: *2017 IEEE 30th Canadian Conference on Electrical and Computer Engineering (CCECE)*. 2017, pp. 1–5. DOI: [10.1109/CCECE.2017.7946818](https://doi.org/10.1109/CCECE.2017.7946818).
- [9] Ivo Frazão et al. *Cyber-security Modbus ICS dataset*. 2019. DOI: [10.21227/pjff-1a03](https://dx.doi.org/10.21227/pjff-1a03). URL: <https://dx.doi.org/10.21227/pjff-1a03>.
- [10] Majed Al-Asiri e El-Sayed M. El-Alfy. «On Using Physical Based Intrusion Detection in SCADA Systems». In: *Procedia Computer Science* 170 (2020). The 11th International Conference on Ambient Systems, Networks and Technologies (ANT) / The 3rd International Conference on Emerging Data and Industry 4.0 (EDI40) / Affiliated Workshops, pp. 34–42. ISSN: 1877-0509. DOI: <https://doi.org/10.1016/j.procs.2020.03.007>. URL: <https://www.sciencedirect.com/science/article/pii/S1877050920304294>.
- [11] Panagiotis Radoglou-Grammatikis et al. «DIDEROT: an intrusion detection and prevention system for DNP3-based SCADA systems». In: *Proceedings of the 15th International Conference on Availability, Reliability and Security*. ARES '20. Virtual Event, Ireland: Association for Computing Machinery, 2020. ISBN: 9781450388337. DOI: [10.1145/3407023.3409314](https://doi.org/10.1145/3407023.3409314). URL: <https://doi.org/10.1145/3407023.3409314>.
- [12] Jakapan Suaboot et al. «A Taxonomy of Supervised Learning for IDSs in SCADA Environments». In: *ACM Comput. Surv.* 53.2 (apr. 2020). ISSN: 0360-0300. DOI: [10.1145/3379499](https://doi.org/10.1145/3379499). URL: <https://doi.org/10.1145/3379499>.
- [13] Mohamed Amine Ferrag et al. «Cyber Security Intrusion Detection for Agriculture 4.0: Machine Learning-Based Solutions, Datasets, and Future Directions». In: *IEEE/CAA Journal of Automatica Sinica* 9.3 (2022), pp. 407–436. DOI: [10.1109/JAS.2021.1004344](https://doi.org/10.1109/JAS.2021.1004344).
- [14] Manar Alanazi, Abdun Mahmood e Mohammad Javed Morshed Chowdhury. «SCADA vulnerabilities and attacks: A review of the state-of-the-art and open issues». In: *Computers and Security* 125 (2023), p. 103028. DOI: <https://doi.org/10.1016/j.cose.2022.103028>.

- [15] Mike Da Silva et al. «PLC Logic-Based Cybersecurity Risks Identification for ICS». In: *Proceedings of the 18th International Conference on Availability, Reliability and Security*. ARES '23. Benevento, Italy: Association for Computing Machinery, 2023. ISBN: 9798400707728. DOI: [10.1145/3600160.3605067](https://doi.org/10.1145/3600160.3605067). URL: <https://doi.org/10.1145/3600160.3605067>.
- [16] Djibrilla Amadou Kountche et al. «The PRECINCT Ecosystem Platform for Critical Infrastructure Protection: Architecture, Deployment and Transferability». In: *Proceedings of the 19th International Conference on Availability, Reliability and Security*. ARES '24. Vienna, Austria: Association for Computing Machinery, 2024. ISBN: 9798400717185. DOI: [10.1145/3664476.3670437](https://doi.org/10.1145/3664476.3670437). URL: <https://doi.org/10.1145/3664476.3670437>.
- [17] Siqu Bai. «Anomaly Detection of Electrical Load in Power Systems Based on Graph Neural Networks». In: *Proceedings of the 2024 9th International Conference on Cyber Security and Information Engineering*. ICCSIE '24. Association for Computing Machinery, 2024, pp. 529–534. ISBN: 9798400718137. DOI: [10.1145/3689236.3689283](https://doi.org/10.1145/3689236.3689283). URL: <https://doi.org/10.1145/3689236.3689283>.
- [18] Neil Ortiz, Alvaro A. Cardenas e Avishai Wool. «SCADA World: An Exploration of the Diversity in Power Grid Networks». In: *Proc. ACM Meas. Anal. Comput. Syst.* 8.1 (feb. 2024). DOI: [10.1145/3639036](https://doi.org/10.1145/3639036). URL: <https://doi.org/10.1145/3639036>.
- [19] Ondrej Pospisil e Radek Fujdiak. «Identification of industrial devices based on payload». In: *Proceedings of the 19th International Conference on Availability, Reliability and Security*. ARES '24. Vienna, Austria: Association for Computing Machinery, 2024. ISBN: 9798400717185. DOI: [10.1145/3664476.3670462](https://doi.org/10.1145/3664476.3670462). URL: <https://doi.org/10.1145/3664476.3670462>.
- [20] Nesibe Yalçın, Semih Çakır e Sibel Ünaldı. «Attack Detection Using Artificial Intelligence Methods for SCADA Security». In: *IEEE Internet of Things Journal* 11.24 (2024), pp. 39550–39559. DOI: [10.1109/JIOT.2024.3447876](https://doi.org/10.1109/JIOT.2024.3447876).